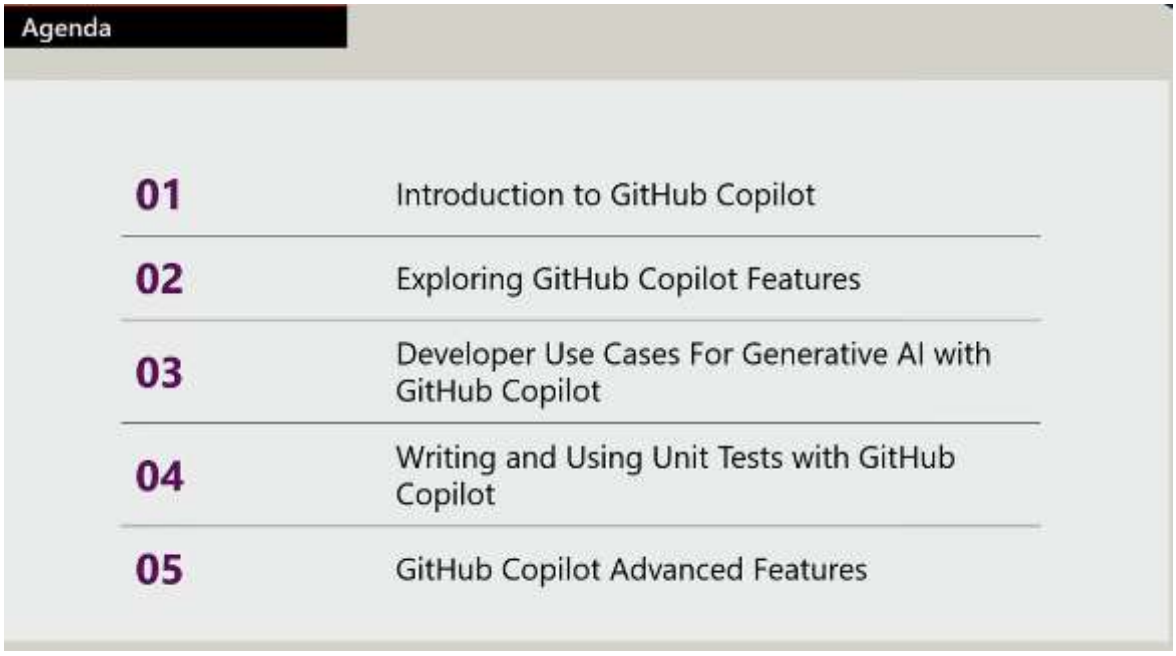
YouTube **0-Course Preview | GH-300 | Episode 0**

https://youtu.be/zzmKB6GL-kA

YouTube **1-Introduction to GitHub Copilot | GH-300 | Episode 1**

https://youtu.be/p43NN4fVREs



01 – Introduction to GitHub Copilot

02 – Exploring GitHub Copilot Features

03 – Developer Use Cases for Generative AI with GitHub Copilot

04 – Writing and Using Unit Tests with GitHub Copilot

05 – GitHub Copilot Advanced Features

GitHub Copilot - Certification

This certification is intended for individuals who have a foundational understanding of GitHub Copilot as a product and its available features, along with hands-on experience in optimizing software development workflows using GitHub Copilot. Once achieved, the cert is valid for 2 years.

- Domain 1: Responsible AI 7%
- Domain 2: GitHub Copilot plans and features 31%
- Domain 3: How GitHub Copilot works and handles data 15%
- Domain 4: Prompt crafting and Prompt engineering 9%
- Domain 5: Developer use cases for AI 14%
- Domain 6: Testing with GitHub Copilot 9%
- Domain 7: Privacy fundamentals and context exclusions 15%

Module 1: Introduction to GitHub Copilot

Introduction

In this module, you'll explore how GitHub Copilot can enhance productivity and security for any organization, understand the features across GitHub Copilot Pro, Business and Enterprise and learn how to enable it. We'll also discuss the opportunities, risks and responsibilities in using AI.

At the end of this module, you'll be able to:

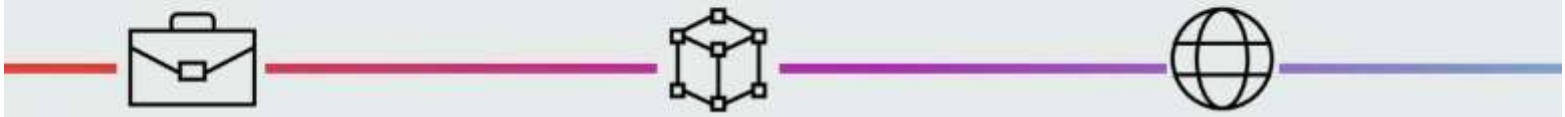
- Understand the feature difference between GitHub Copilot Pro, Business and Enterprise.
- GitHub Copilot Use Cases and Customer Stories
- Setup, Configure and Troubleshoot GitHub Copilot
- Opportunities and Risks of using AI, and the Microsoft/GitHub Responsible AI Framework

Explore GitHub Copilot Plan Options

- GitHub Copilot Enterprise includes collaborative chat within pull requests, pull request summaries, docset management, and code review.
- Enterprise and Business plans offer zero data retention for code snippets, organization-wide policy management, and VPN proxy support, with Enterprise providing comprehensive security tool integration.
- All plans integrate with your editor and live on GitHub.com, but only Enterprise and Business plans offer advanced management features.

	GitHub Copilot Enterprise	GitHub Copilot for Business	GitHub Copilot Pro & Free
Collaborative Chat within pull requests	✓	×	×
Lives on Github.com	✓	✓	✓
Plugs right into your editor	✓	✓	✓
Pull request summaries	✓	✓	✓
Copilot docset management	✓	×	×
Copilot code review	✓	×	✓
Zero data retention for code snippets and usage telemetry	✓	✓	×
Organization-wide policy management	✓	✓	×
Integration with security tools	✓	Limited	×
Audit logging and reporting	✓	Limited	×
VPN proxy support via self-signed certificates	✓	✓	×

GitHub Copilot use cases and customer stories



Faster Coding

- **Increased Speed**: Nearly 90% of developers complete tasks faster with GitHub Copilot.
- **Time Savings**: Coyote Logistics saved significant time on tasks like writing Terraform configurations.
- **Efficient Details**: GitHub Copilot fills in details after the base configuration is written.

Improved Flow

- **Reduced Disruptions**: 73% of developers said Copilot helps them stay in the flow longer.
- **Mental Energy**: 87% of developers preserved mental energy when working through repetitive tasks.
- **Focus Maintenance**: Helps developers stay focused on solving complex business challenges.

Enhanced Productivity

- **Context Switching**: Duolingo reduced context switching and manual boilerplate code production.
- **New Developers**: Estimated 25% speed increase for developers new to a repository or framework.

GitHub Copilot, your AI pair programmer

Copilot for Chat

- **Chat Interface**: Integrates with VS Code and Visual Studio.
- **Error Recognition**: Identifies code errors and suggests fixes, generating unit tests.

Copilot for Pull Requests

- **Iterate and Validate**: Integrate suggested changes to code by using Copilot Workspace.
- **Review and Modify**: Developers can review and adjust the suggested descriptions.

Copilot for the CLI

- **Command Composition**: Helps compose commands and loops, making terminal usage more efficient.
- **Syntax Assistance**: Provides obscure find flags and command syntax reminders.

Subscription Plans

- **Personal Accounts**: Available through GitHub Copilot Free and Pro.
- **Organization and Enterprise Accounts**: Available through GitHub Copilot Business and Enterprise.

GitHub Copilot Pro+

- Highest level of access for individual developers.
- Includes Copilot coding agent

GitHub Copilot Business

- **Access Control**: Allows organizations to manage who can use GitHub Copilot.
- **Productivity and Security**: Features include code completions, chat in IDE and mobile, and security filters.

GitHub Copilot Enterprise

- **Enhanced Features**: Includes all Business features plus personalized suggestions based on internal code.
- **Integration and Documentation**: Provides chat interface integration and helps build documentation.

How to get started

Your GitHub Organization owner can enable GitHub Copilot for the organizations by first establishing the policy and then assigning users.

Enforce a Policy

- **Access Policies**: In the enterprise sidebar, select Policies, then Copilot, and configure access for your subscription.
- **Manage Access**: Under Manage organization access to GitHub Copilot, set the desired access level.

Enable Access for All Users

- **Organization Settings**: Select your profile photo, then Your organizations, and navigate to Settings.
- **User Permissions**: In the Copilot Access section, select Allow for all members, confirm seat assignment, and save changes.

Enable Access for Selected Users

- **Selective Access**: Follow the enabling steps, then select Selected teams/users under User permissions.
- **Confirm and Save**: Confirm seat assignment for selected users and save changes.

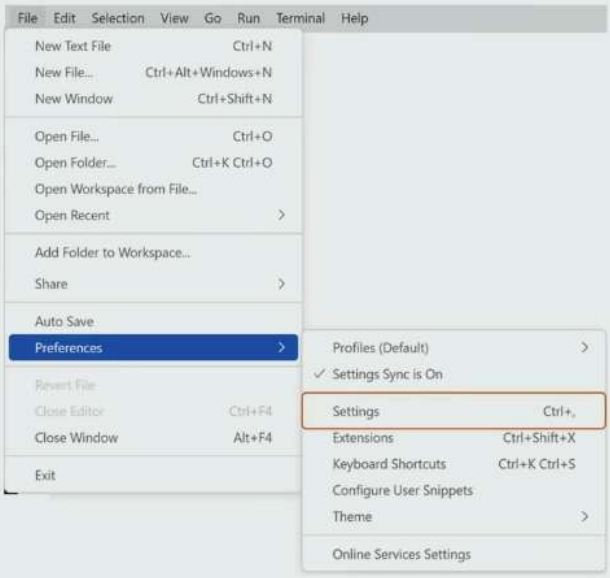
Disable Access for the Organization

- **Disable Access**: Follow the enabling steps, then select Disabled under User permissions.
- **Confirm and Save**: Confirm the decision to disable access for all users and save changes.

Set up GitHub Copilot

Before you can start using GitHub Copilot, you need to set up a free trial or subscription for your account.

- **Installation:** Install the GitHub Copilot extension from the Visual Studio Marketplace, open VS Code, and sign in to GitHub if prompted.
- **Enable/Disable:** Use the status icon in the bottom pane of VS Code to enable or disable GitHub Copilot globally or for specific languages.
- **Inline Suggestions:** Configure inline suggestions by navigating to Preferences > Settings > Extensions > GitHub Copilot, and selecting or clearing the checkbox for auto completions.



Interact with Copilot

- **Real-Time Code Completions:** Copilot offers inline suggestions as you type, predicting your next steps and displaying them subtly.
- **Accepting and Rejecting Suggestions:** Use the Tab or right arrow key to accept suggestions, and the Esc key to reject them. Keyboard shortcuts:
  macOS: Command+→
  Windows or Linux: Control+→
- **Command Palette Access:** Quickly access Copilot functions with **Ctrl+Shift+P** (Windows/Linux) or **Cmd+Shift+P** (Mac) to perform tasks like generating unit tests.
- **Interactive Copilot Chat:** Communicate with Copilot using natural language to ask questions or request code snippets directly in your IDE.
- **Context-Specific Inline Chat:** Request code modifications or explanations without switching contexts using **Ctrl+I** (Windows/Linux) or **Cmd+I** (Mac).
- **Comments to Code:** Convert comments into code using natural language processing, making it easy to draft straightforward tasks quickly.
- **Multiple Code Suggestions:** Explore different coding approaches by cycling through multiple alternatives using the light bulb icon or **Alt+/Option+]**.
- **Automated Test Generation:** Generate unit tests for functions or classes to ensure code quality and reliability, saving time and effort.

Demo-Setup

Demonstration

Set up GitHub Copilot

In this demonstration, you'll see how to:

- **Define GitHub Copilot settings in GitHub**
- **Install and Validate GitHub Copilot for VS Code**
- **Install and Validate GitHub Copilot for Visual Studio**

Manage content exclusions

The content exclusion feature in GitHub Copilot helps protect sensitive information by preventing the use of specific files, directories, or repositories to inform code-completion suggestions.

Configuring Content Exclusions for Repositories

- **Repository Settings**: Go to the repository's main page, select Settings, and specify files or directories to exclude under the Copilot section.
- **Exclusion Paths**: Use the Repositories and paths to exclude section to define exclusions.

Configuring Content Exclusions for Organizations

- **Organization Settings**: Access your organization's settings from your profile photo, then navigate to Copilot > Content exclusion.
- **Exclusion Details**: Enter the details of files or repositories to exclude from Copilot suggestions.

Impact of Content Exclusions

- **Code Completion**: Excluded files won't have code completion available, and their content won't inform suggestions in other files.
- **Chat Responses**: Excluded content won't be used in GitHub Copilot Chat responses, affecting the quality and relevance of suggestions.

Limitations of Content Exclusions

- **IDE Limitations**: Exclusions might not apply in some IDE features, like Copilot Chat in Visual Studio Code.

Troubleshoot common problems with GitHub Copilot

The absence of code suggestions when using Copilot can be solved by trying these troubleshooting actions:

Missing Code Suggestions

- **Internet and Updates**: Ensure a stable internet connection and update the GitHub Copilot extension.
- **IDE Compatibility**: Verify that your IDE is compatible and properly configured for GitHub Copilot.

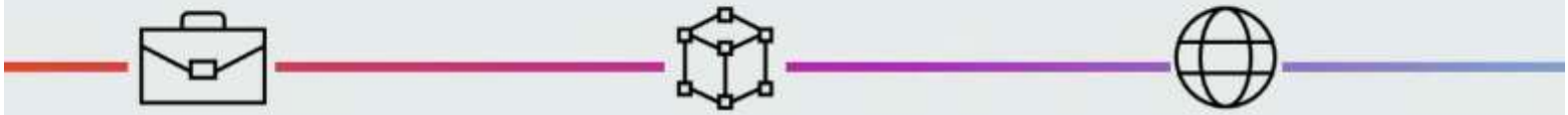
Addressing Content Exclusion Issues

- **Delayed Application**: Reload settings in your IDE to apply content exclusions immediately.
- **Scope and Limitations**: Ensure all team members have the correct settings and be aware of IDE-specific limitations.

Improving Unsatisfactory Code Suggestions

- **Clear Context**: Provide descriptive comments and meaningful variable names.
- **Prompt Techniques**: Use specific commands and experiment with prompt lengths for better suggestions.

Mitigate AI risks



Opportunities and Risks of AI

- **Innovation and Efficiency**: AI offers significant potential for driving innovation and improving efficiency across various sectors.
- **Risks**: AI systems can lead to unintended consequences such as privacy violations and lack of transparency.

Mitigating AI Risks

- **Governance Frameworks**: Implementing robust governance frameworks is essential to manage AI risks effectively.
- **Transparency and Oversight**: Ensuring transparency in AI processes and incorporating human oversight can help mitigate potential negative impacts.

Responsible AI Principles

- **Ethical Development**: Responsible AI focuses on developing AI systems in a safe, trustworthy, and ethical manner.
- **Human-Centered Design**: Keeping human goals at the center of AI system design decisions ensures fairness, reliability, and transparency.

Microsoft and GitHub's five principles of responsible AI

- **Fairness**: AI systems should treat all people fairly, avoiding differential impacts on similarly situated groups.
- **Reliability and Safety**: AI systems must operate reliably, safely, and consistently, minimizing unintended harm and perform as intended.
- **Privacy and Security:** Protecting user privacy and data security is crucial, involving consent, minimal data collection, anonymization, and strong encryption.
- **Inclusiveness**: AI systems should be accessible and empower everyone, ensuring they work well for diverse users and are available worldwide.
- **Transparency and Accountability**: AI systems must be understandable, with clear explanations, auditability, and continuous monitoring to ensure responsible operation.

Explore contractual protections in GitHub Copilot and disabling matching public code

To help ensure that your organization remains compliant with legal requirements, GitHub Copilot offers:

IP indemnity: The GitHub Copilot Business and Enterprise plans include IP indemnity, which provides legal protection against intellectual property claims related to the use of Copilot suggestions.

- With IP indemnity, if any suggestion from GitHub Copilot is challenged as infringing on third-party IP rights, GitHub assumes legal responsibility.

Data Protection Agreement (DPA): GitHub offers a DPA that outlines the measures taken to protect your data and ensure compliance with data privacy regulations.

- These agreements provide transparency and assurance that your data is handled securely and responsibly.

GitHub Copilot Trust Center: The GitHub Copilot Trust Center provides detailed information about how GitHub Copilot works, including security, privacy, compliance, and intellectual property safeguards.

- This resource helps organizations feel confident using GitHub Copilot while adhering to best practices and legal requirements.

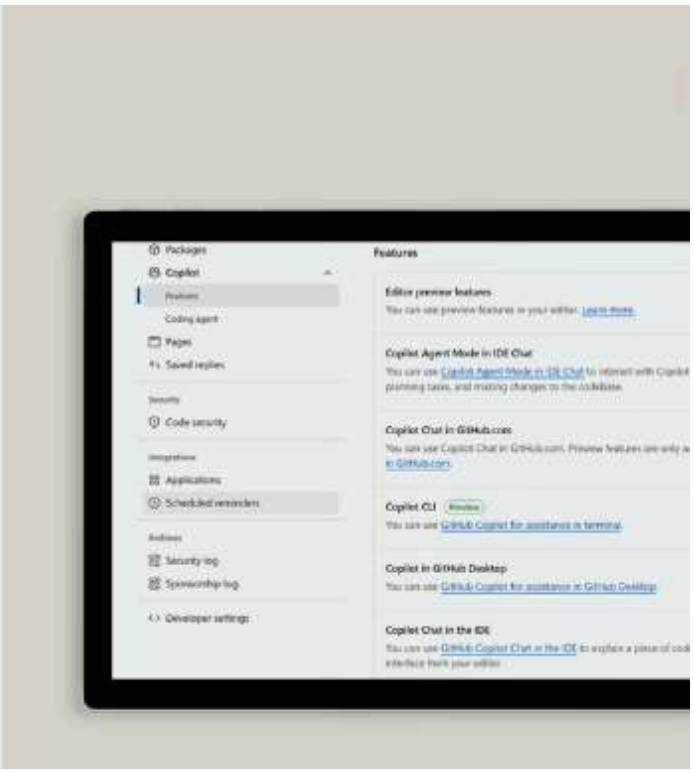
Demo-ResponsibleAI

Demonstration

GitHub Copilot Responsible AI

In this demonstration, you'll see how to:

- **Explore GitHub Copilot Trust Center**
- **Managing GitHub Copilot policies**



https://github.com/trust-center

https://copilot.github.trust.page/

https://copilot.github.trust.page/media

Summary

In this module, we explored how GitHub Copilot can enhance productivity and security for any organization. We discussed the difference in features between GitHub Copilot Pro, Business and Enterprise and you learned how to enable it, configure and troubleshoot it.

We also discussed the opportunities, risks and responsibilities in using AI.

You are now able to:

- Identify the feature difference between GitHub Copilot Pro, Business and Enterprise.
- Describe GitHub Copilot Use Cases and Customer Stories
- Setup, Configure and Troubleshoot GitHub Copilot
- Understand the opportunities and risks in using AI and what the details are of Microsoft/GitHub Responsible AI Framework

2-Exploring GitHub Copilot features | GH-300 | Episode 2

https://youtu.be/7iuQ4iOxq18

Introduction

In this module you'll learn:

- Explore GitHub Copilot plans and their associated management and customization features
- How to use GitHub Copilot in a development environment like Visual Studio Code and the command line
- Prompt engineering foundations, best practices, and process flow
- How Copilot uses Large Language Models (LLMs)
- Control how Copilot handles your data
- Troubleshoot common problems with GitHub Copilot

Explore GitHub Copilot plans and their associated management and customization features

Management Policies: GitHub Copilot offers different features across Free, Pro, Business, and Enterprise plans, including public code filters and user management.

Security and Privacy: Business and Enterprise plans provide robust privacy controls, such as excluding specific files from analysis, accessing detailed audit logs.

Plan Selection Considerations: Key factors include data privacy, policy management, data collection and retention, and IP indemnity, which are crucial for compliance and security.

Feature	Free* & Pro	Business	Enterprise
Public code filter	✓	✓	✓
User management	✗	✓	✓
Data excluded from training by default	✓	✓	✓
Enterprise-grade security	✗	✓	✓
IP indemnity	✗	✓	✓
Content exclusions	✗	✓	✓
SAML SSO authentication	✗	✓	✓
Require GitHub Enterprise Cloud	✗	✗	✓
Usage metrics	✗	✓	✓

Code completion with GitHub Copilot

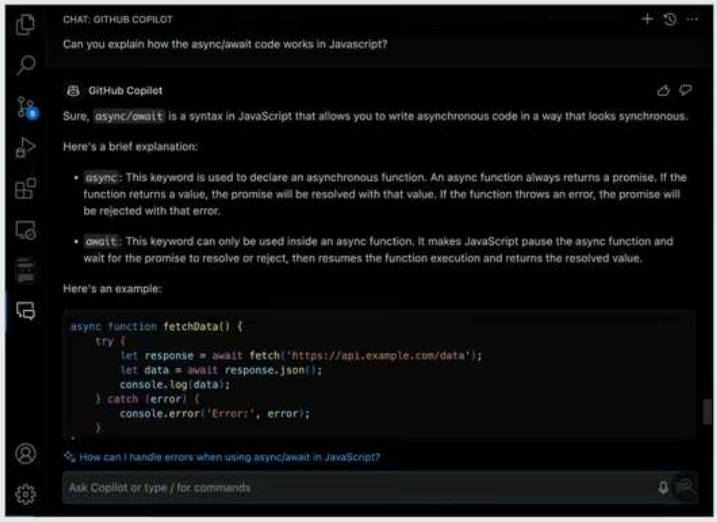
Copilot offers code suggestions as you type: sometimes completing the current line, sometimes suggesting a whole new block of code.

- **Supported Languages**: GitHub Copilot excels in Python, JavaScript, TypeScript, Ruby, Go, C#, and C++, but also supports many other languages and frameworks.
- **Auto Suggestions**: Provides real-time, context-aware code suggestions, completing lines or blocks of code to save development time.
- **Multiple Suggestions Pane**: Offers multiple code suggestions, allowing you to explore different options using the control panel or keyboard shortcuts.
- **Adaptability**: Adapts to your coding style, including method implementation, naming conventions, formatting, comment style, and design patterns.
- **Using Comments**: Enhances code suggestions by understanding and incorporating coding comments through natural language processing and contextual analysis.
- **Efficiency**: Helps streamline coding tasks by reducing the need to search for syntax or troubleshoot logic.

GitHub Copilot Chat

GitHub Copilot Chat allows developers to have natural language conversations about their code, ask questions, and receive intelligent responses and suggestions in real-time.

- **Interactive AI Assistant**: GitHub Copilot Chat allows natural language conversations within the development environment for real-time coding assistance.
- **Code Generation and Debugging**: Helps generate complex code, debug errors, and provide step-by-step explanations.
- **Code Explanations**: Breaks down complex code snippets, explains functionality, and offers optimization insights.
- **Slash Commands and Agents**: Use slash commands and custom agents to perform specific actions and improve prompt effectiveness.



Demo-Chat

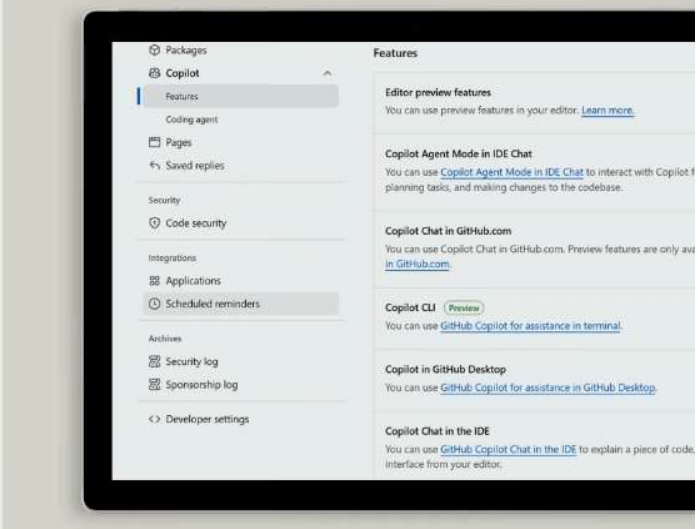
Demonstration: GitHub Copilot Chat

Demonstration

GitHub Copilot Chat

In this demonstration, you'll see how to:

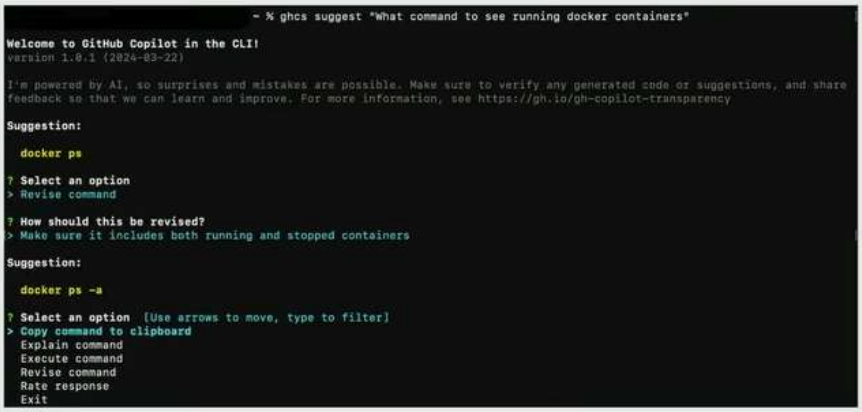
- **Use GitHub Copilot Chat**
- **Use GitHub Copilot Inline Chat**
- **Switch LLMs**



GitHub Copilot for the Command Line

GitHub Copilot Chat allows developers to have natural language conversations about their code, ask questions, and receive intelligent responses and suggestions in real-time.

- **Command-Line Integration**: GitHub Copilot enhances command-line workflows by providing explanations, suggesting commands, and executing them.
- Offers frequently used commands like explaining "sudo apt-get" or suggesting commands to undo the last commit.
- Simplifies complex tasks and improves productivity for both new and seasoned command-line users.



See: gh copilot suggest

Demo-CLI

Demonstration: GitHub Copilot CLI

Demonstration

GitHub Copilot CLI

In this demonstration, you'll see how to:

- **Use GitHub Copilot @Terminal integration**
- **Using GitHub Copilot CLI**

Prompt engineering foundations and best practices

What is Prompt Engineering?

- Craft clear instructions to guide AI systems like GitHub Copilot.
- Ensures code is syntactically, functionally, and contextually correct.
- Generates context-appropriate code tailored to project needs.

Principles of Prompt Engineering

- Focus on a single, well-defined task or question for clarity.
- Provide explicit and detailed instructions for precise code suggestions.
- Keep prompts concise to maintain clarity without overloading the AI.

Best Practices in Prompt Engineering

- Use descriptive filenames and keep related files open for rich context.
- **Iterate**: Refine prompts based on feedback to improve AI responses.
- **Contextualize**: Provide relevant background information to enhance code suggestions.

Advanced Strategies

- **Layered Prompts**: Break complex tasks into smaller, manageable prompts.
- **Feedback Loop**: Continuously evaluate and adjust prompts for optimal performance.
- **Collaborative Input**: Involve team members to ensure comprehensive and effective prompts.

Prompt → Context
→ Goal
→ Persona
Most accurate Statistical Anasys

Prompt engineering advanced strategies

Role Definition (Role Priming)

- **Example**:Weak prompt: “Explain Kubernetes”.
- **”Role-primed prompt**: “You are a senior cloud architect mentoring a junior engineer. Explain Kubernetes using practical analogies and real-world deployment scenarios.

- **Structured Prompt with Constraints**
  - **Structured prompt**: “Summarize this article in exactly three bullet points: • One about the problem • One about the solution • One about the impact Keep each bullet under 12 words

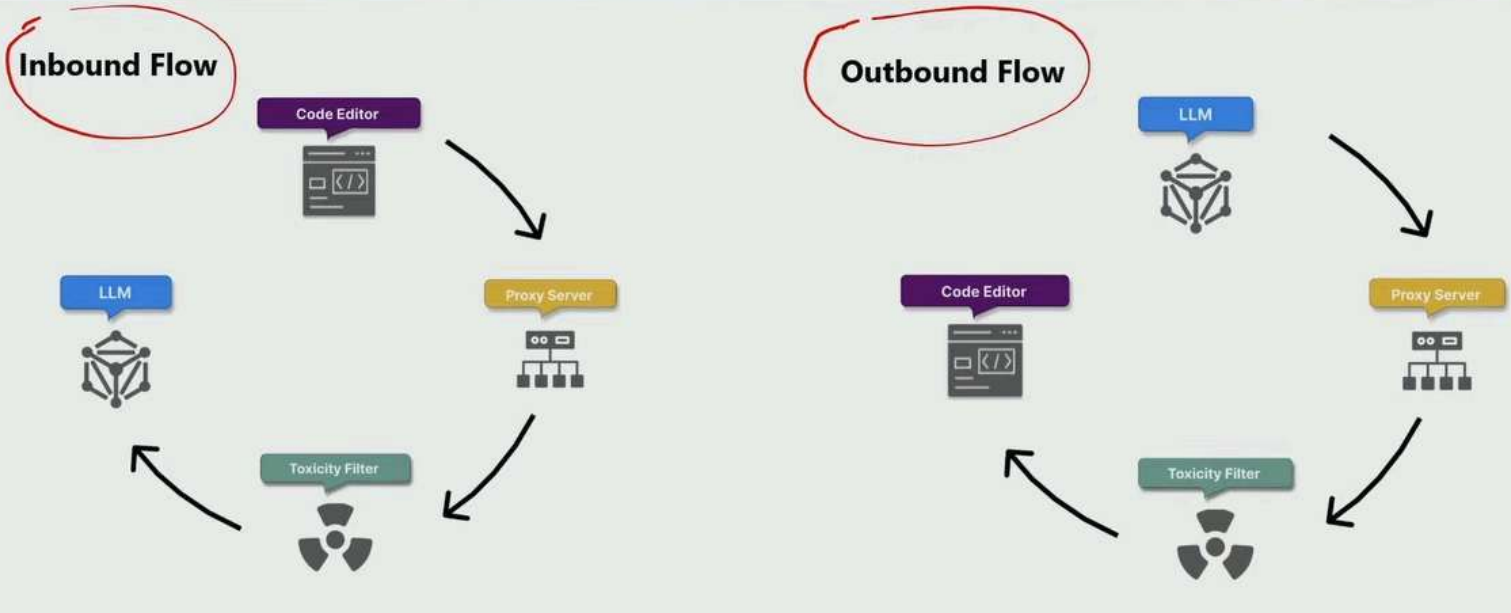
Chain of Thought

- **Example**: “Break down your reasoning into three steps:1) Identify the core issue. 2) Evaluate possible approaches. 3) Recommend the best option with justification
- Then provide only the final recommendation in your answer

- **Micro-Tuning**
  - First attempt: “Write a poem about cloud computing.”
  - Micro-tuned: “Write a short, metaphor-driven poem about cloud computing that uses natural imagery and ends with a hopeful tone.”

GitHub Copilot user prompt process flow

GitHub Copilot receives prompts and returns code suggestions or responses in its data flow. This process suggests an inbound and outbound flow.



GitHub Copilot data

Data Handling for Code Suggestions

- **No Retention**: GitHub Copilot in the code editor discards prompts once a suggestion is returned.
- **Opt-Out Option**: Individual subscribers can opt-out of sharing their prompts for model fine-tuning.

Data Handling for Copilot Chat

- **Response Formatting**: Formats responses for optimal presentation, highlighting code snippets.
- **User Engagement**: Allows follow-up questions and maintains conversation history for context.
- **Data Retention**: Retains prompts and suggestions for 28 days outside the code editor.

Prompt Types Supported by Copilot Chat

- **Direct Questions**: Ask specific coding questions or troubleshoot issues.
- **Code-Related Requests**: Request code generation, modification, or explanations.
- **Open-Ended Queries**: Explore coding concepts or seek general guidance.

Contextual Prompts

- **Code Snippets**: Provide snippets for tailored assistance.
- **Scenario Descriptions**: Describe coding scenarios for specific help.

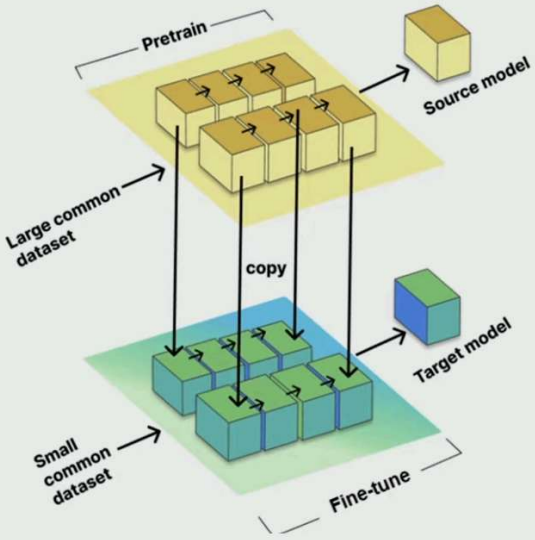
Demo-Prompt&Data

Demonstration: GitHub Copilot Prompts and Data

GitHub Copilot Large Language Models (LLMs)

Fine-tuning LLMs involves training the model on a smaller, task-specific dataset, known as the target dataset, while using the knowledge and parameters gained from the source model.

- **Volume of Training Data**: LLMs are trained on vast amounts of text from diverse sources, providing a broad understanding of language and context.
- **Contextual Understanding**: They generate contextually relevant and coherent text, making meaningful contributions to various forms of communication.
- **Machine Learning Integration**: LLMs are neural networks with millions or billions of parameters, fine-tuned to understand and predict text effectively.
- **Versatility**: These models can be tailored for specialized tasks, making them highly versatile across different domains and languages.

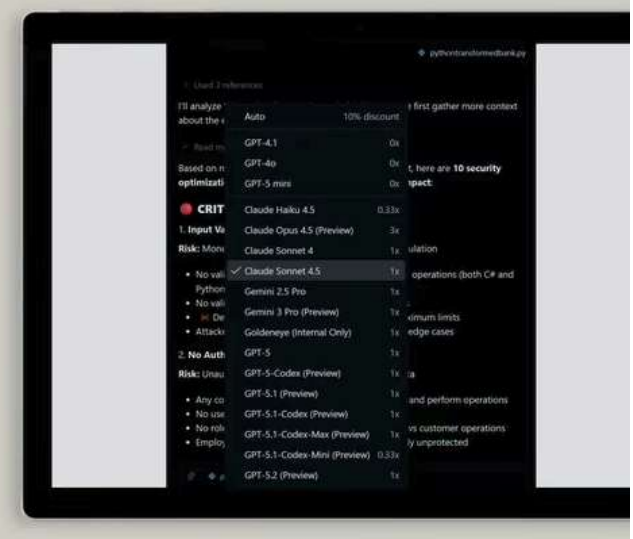


Demonstration

GitHub Copilot LLMs

In this demonstration, you'll see how to:

- **Use GitHub Copilot LLMs in Chat and Inline Chat**
- **Enable/Disable GitHub Copilot LLMs**



Manage content exclusions

The content exclusion feature in GitHub Copilot helps protect sensitive information by preventing the use of specific files, directories, or repositories to inform code-completion suggestions.

Configuring Content Exclusions for Repositories

- **Repository Settings**: Go to the repository's main page, select Settings, and specify files or directories to exclude under the Copilot section.
- **Exclusion Paths**: Use the Repositories and paths to exclude section to define exclusions.

Configuring Content Exclusions for Organizations

- **Organization Settings**: Access your organization's settings from your profile photo, then navigate to Copilot > Content exclusion.
- **Exclusion Details**: Enter the details of files or repositories to exclude from Copilot suggestions.

Impact of Content Exclusions

- **Code Completion**: Excluded files won't have code completion available, and their content won't inform suggestions in other files.
- **Chat Responses**: Excluded content won't be used in GitHub Copilot Chat responses, affecting the quality and relevance of suggestions.

Limitations of Content Exclusions

- **IDE Limitations**: Exclusions might not apply in some IDE features, like Copilot Chat in Visual Studio Code.

Summary ch02

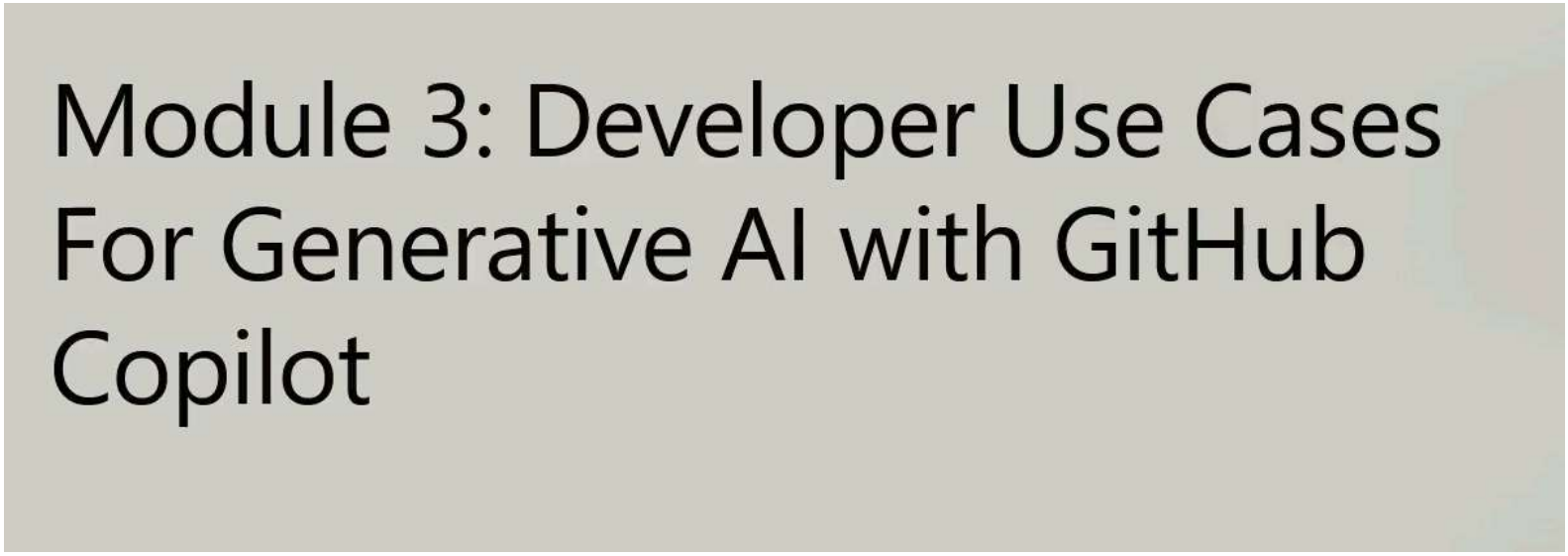
Summary

Now that you've finished this module, you should be able to describe:

- Explore GitHub Copilot plans and their associated management and customization features
- How to use GitHub Copilot in a development environment like Visual Studio Code and the command line
- Prompt engineering foundations, best practices, and process flow
- How Copilot uses Large Language Models (LLMs)
- Control how Copilot handles your data

3-Developer use cases for generative AI with GitHub Copilot | Episode 3

https://youtu.be/cD10ky-ndQ4 **42min**



Introduction

In this module, we will explore various developer use cases for GitHub Copilot, examining how it enhances productivity, customize a JavaScript project with GitHub Copilot in GitHub Codespaces and modify a project by using a prompt to customize a Python API

In this module you'll:

- Identify specific ways GitHub Copilot integrates seamlessly into developer workflows, enhancing the overall development experience and supporting individual coding preferences.
- Craft prompts to generate suggestions from GitHub Copilot.
- Configure a GitHub repository in Codespaces and install GitHub Copilot extension.

Boost developer productivity with AI

GitHub Copilot helps bridge the gap between learning and actual implementation with the following:

Accelerate Learning New Languages and Frameworks

- **Code Suggestions**: Provides context-aware snippets to illustrate the usage of unfamiliar functions and libraries.
- **Language Support**: Supports a wide range of languages, aiding smooth transitions.

Minimizing Context Switching

- **In-Editor Assistance**: Offers code suggestions directly in the IDE, reducing the need to search for solutions online.

Enhanced Documentation Writing

- **Inline Comments**: Generates relevant comments explaining complex code sections.
- **Function Descriptions**: Suggests descriptions for functions, including parameter explanations and return value details.

Improving Code Consistency

- **README Generation**: creates README files with structured content based on the codebase.
- **Documentation Consistency**: Helps maintain a consistent documentation style across the project.

Align with developer preferences

Code Generation and Completion

- **Multiple Suggestions**: Provides various code suggestions for ambiguous scenarios, allowing developers to choose the best option.
- **Language-Specific Idioms**: Suggests idiomatic code and best practices for different programming languages.

Writing Unit Tests and Documentation

- **Test Case Generation**: Suggests relevant test cases based on function signatures and behavior, including edge cases.
- **Documentation Stubs**: Generates initial documentation stubs for functions, classes, and modules, which can be refined by developers.

Code Refactoring

- **Pattern Recognition**: Identifies common patterns and suggests more efficient or cleaner alternatives.
- **Modern Syntax Suggestions**: Recommends modern language features for evolving languages like JavaScript ECMAScript.

Debugging Assistance

- **Error Explanation**: Provides plain-language explanations for error messages and suggests potential fixes.
- **Log Statement Generation**: Suggests relevant log statements to help diagnose issues in complex code paths.

Demo-Developer

Demonstration: GitHub Copilot Developer Use Cases

Comments:

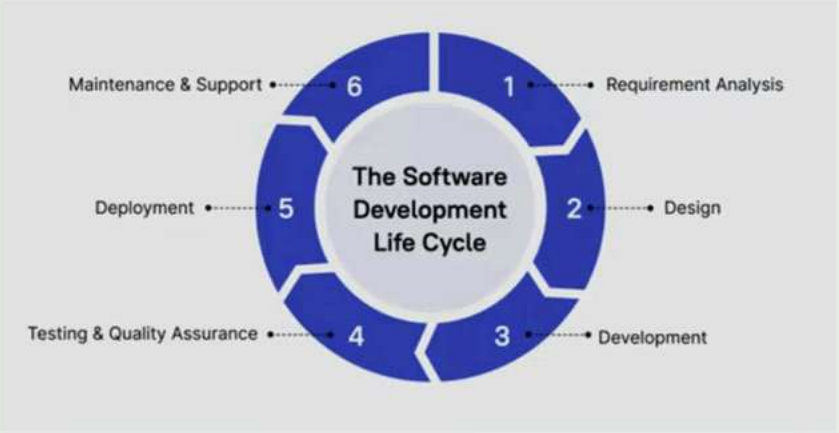
"Add a search bar above the activities list that filters activities in real-time as the user types. Search should match activity name, description, or schedule."

@workspace help me transforming code from python to dotnet, preferably the latest framework. If possible, integrate the existing css if it doesnt break the app

AI in the Software Development Lifecycle (SDLC)

GitHub Copilot enhances different SDLC phases, from initial planning to deployment and maintenance.

- **Boilerplate Code and Design Patterns**: Generates repetitive code structures and suggests appropriate design patterns.
- **Code Optimization and Translation**: Offers efficient code alternatives and assists in translating code between languages.
- **Unit Tests and Edge Cases**: Generates test cases and suggests scenarios covering edge cases.
- **Bug Fixes and Refactoring**: Proposes fixes for issues and suggests improvements to keep the codebase modern and efficient.



Demo-Devops

Demonstration: GitHub Copilot DevOps Use Cases

Demonstration

GitHub Copilot DevOps Use Cases

In this demonstration, you'll see how to:

- **Generate DevOps Code Snippets (e.g. Cloud CLI, Bash, PowerShell, DockerFile, Cloud IAC Templates, CI/CD Pipelines,...)**

Comments:

The app is working as expected. We need to bring this now into our devops team, and perform the following steps.
A) run it locally in a container,
B) push the container image to Azure, using Azure CLI instructions.
C) automate the full build and deployment using multistage CI/CD pipeline in GitHub Actions

Understand limitations and measure impact

Code Quality & Correctness

- **Potential for Errors:** Copilot may suggest code with bugs or that doesn't fully meet requirements.
- **Security Concerns:** Generated code might not adhere to best security practices, requiring careful review.

Language & Framework Specificity

- **Varying Performance:** Effectiveness can vary across different programming languages and frameworks.
- **Niche Technologies:** Suggestions may be less accurate or relevant for less common or newer technologies.

Complex Problem Solving

- **High-Level Design Limitations:** Copilot excels at code-level tasks but may not grasp complex architectural decisions.
- **Creativity Constraints:** While helpful, Copilot cannot replace human creativity in solving novel problems.

Summary ch03

Summary

- Explored GitHub Copilot's capabilities and its impact on the development process.
- Learned how GitHub Codespaces improves the software development lifecycle by enabling customization and live suggestions.
- Configured a GitHub repository in Codespaces and installed the GitHub Copilot extension.
- Understood how Copilot assists with coding through autocomplete-style suggestions.
- Applied prompt engineering best practices to generate effective AI suggestions.
- Used GitHub Copilot Chat to ask and receive coding-related questions.
- Moving forward, developers should:
  - Continuously explore new ways to integrate GitHub Copilot into workflows.
  - Regularly assess the impact of AI-assisted coding on development processes.
  - Maintain a balance between leveraging AI assistance and fostering human creativity and problem-solving skills.

4-Writing and using unit tests with GitHub Copilot | GH-300 | Episode 4

https://youtu.be/XSJO3nUEQ2c **20min**

00:06 Writing and Using Unit Tests with GitHub Copilot

Introduction

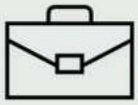
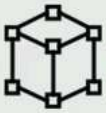
By the end of this module, you're able to use GitHub Copilot and Visual Studio Code tools to efficiently create, manage, and run unit tests. Using GitHub Copilot improves the reliability of your codebase and saves valuable development time.

In this module you'll be:

- Creating unit tests in Visual Studio Code using GitHub Copilot and GitHub Copilot Chat.
- Creating specific unit tests for edge cases and boundary conditions using GitHub Copilot Chat.
- Creating unit test projects and managing unit tests in Visual Studio Code.
- Completing a "create unit tests" challenge and reviewing a possible solution.

IMPORTANT: To complete this GitHub Copilot training, you must have an active subscription for GitHub Copilot in your personal GitHub account, or you must be assigned to a subscription managed by an organization or enterprise.

Examine the unit testing tools and environment


GitHub Copilot Tools in Visual Studio Code

- **Code Line Completions**: Write code more efficiently with real-time suggestions.
- **Inline Chat**: Start conversations directly from the editor for coding help.
- **Smart Actions**: Run tasks without writing prompts, enhancing productivity.

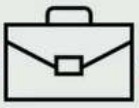
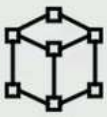
Visual Studio Code Support for Unit Tests

- **Required Resources**: .NET 8.0 SDK or later, C# Dev Kit extension, and a test framework package.
- **Test Explorer**: View all test cases in a tree view.
- **Run/Debug Tests**: Features to run and debug test cases, and view results.

Enabling a Test Framework

- **Command Palette**: Use Ctrl+Shift+P (Windows/Linux) or Cmd+Shift+P (macOS) to open the Command Palette.
- **Project Setup**: Create new test projects and configure test runners using the Command Palette.
- **Framework Support**: Supports xUnit, NUnit, and MSTest for creating and managing unit tests.

Using GitHub Copilot to Generate Unit Tests


GitHub Copilot Tools in Visual Studio Code

- **Use Prompt Engineering**: Run the necessary prompts to get Unit Tests code suggestions
- **Use VS Code Context**: Run @workspace / setuptests @workspace / tests

Visual Studio Code Support for Unit Tests

- **Generate Initial Tests**: GitHub Copilot will prompt you for a Framework if needed and generates code
- **Expand with Edge Cases:** Once code is set up, expand for Unit Tests Edge Cases

Enabling a Test Framework

- **Run and Verify:** Run the actual tests; if C# DotNet, integrate the C# DevKit VS Code extension, to simulate live tests from within the code view

Demo-Unit Test

05:40 Demonstration: GitHub Copilot Unit Tests Development

Demonstration

GitHub Copilot Unit Tests Development

In this demonstration, you'll see how to:

- **Use GitHub Copilot to Generate Unit Tests**
- **C# DevKit for .NET applications Unit Tests Run and Verify**

Comments

for the dotnet project, how to go ahead with unit testing, knowing I'm a junior developer. can you explain the purpose of unit testing, what framework options for dotnet to choose from and how to get started

Summary ch04

Summary

In this module, you learned about the use of GitHub Copilot Chat and Visual Studio Code for creating and managing unit tests.

In this module you:

- Created unit tests in Visual Studio Code using GitHub Copilot and GitHub Copilot Chat.
- Created specific unit tests for edge cases and boundary conditions using GitHub Copilot Chat.
- Created unit test projects and managing unit tests in Visual Studio Code.

5-GitHub Copilot advanced features | GH-300 | Episode 5

https://youtu.be/VbHvd2c_ids **59min**

00:07 GitHub Copilot Advanced Features

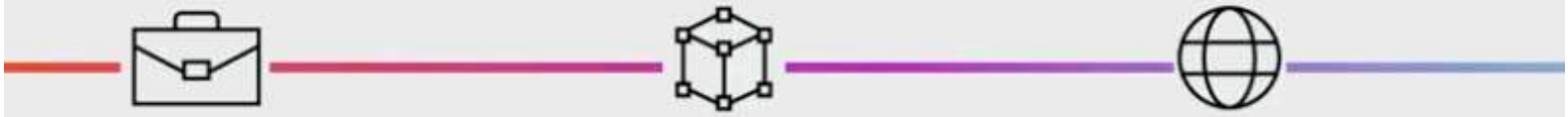
Introduction

In this module, you'll explore how GitHub Copilot enhanced productivity, as well as developer and DevOps team's velocity, by integrating automation using Agent Mode. We also explain the integration of additional knowledge into the Agent, using Model Context Protocol (MCP) and the concept of Agent.md. Last, you will explore how Github Copilot can integrate in your GitHub DevOps processes such as PR Reviews and Actions Pipeline Troubleshooter.

At the end of this module, you'll be able to:

- Understand the capabilities of GitHub Copilot Agent Mode
- Integrate additional knowledge by adding MCP Server connections to your Agents
- Configure and use Agent.md
- PR Reviews using GitHub Copilot
- GitHub Actions Pipeline Troubleshooter

GitHub Copilot Agent Mode



Agent Mode: Purpose

- Enables Copilot to act as an **autonomous agent** that plans, executes, and iterates across multi-step development and DevOps tasks.
- Moves beyond single prompt–response interactions to **goal-oriented outcomes** across repositories, pipelines, and workflows.

Agent Mode: Key Benefits

- **Reduced manual effort** by automating complex, multi-step tasks such as debugging, refactoring, and CI/CD troubleshooting.
- **Improved velocity** Keeping teams in flow while Copilot handles orchestration and analysis.
- **More consistent outcomes** policy-aware, context-driven execution aligned with repository and organization standards.

Agent Mode: Core Characteristics

- **Goal-driven execution**: Translates high-level intents into structured plans and actionable steps.
- **Context-aware reasoning**: Uses repository state, issues, pull requests, and workflows as live context.
- **Iterative feedback loop**: Observes results, adapts actions, and refines solutions automatically.

GitHub Copilot Agent Mode Use Cases

- **Development Scenarios**
  - that extend beyond a single file or prompt and instead require coordinated actions across code, configuration, and automation workflows.
  - It is particularly valuable when engineers need support navigating complex environments where decisions span repositories, pipelines, and operational context rather than isolated code snippets.
- **Debugging and Remediation**
  - In debugging and remediation scenarios, Agent Mode can investigate failing tests or pipelines, trace issues across multiple files, and iteratively propose fixes while validating results.
  - This is especially effective when issues are non-obvious, intermittent, or influenced by configuration, dependencies, or workflow logic rather than a single line of code.
- **DevOps and CI/CD operations**
  - Assists with diagnosing GitHub Actions failures, analyzing workflow definitions, identifying misconfigurations, and suggesting corrections that align with existing pipeline behavior.
  - This reduces the time spent manually inspecting logs and YAML files and accelerates recovery by handling analysis and iteration in a structured way.
- **Collaboration and Scale**
  - supporting PR reviews, enforcing repository conventions, and applying organization-specific standards consistently.

Demo-Agent Mode

04:38 Demonstration: GitHub Copilot Agent Mode

Demonstration

GitHub Copilot Agent Mode

In this demonstration, you'll see how to:

- **Switch from Ask Mode to Agent Mode**
- **Use Prompt Engineering to trigger Agent Mode coding execution**

Comments:The app is working as expected. We need to bring this now into our devops team, and perform the following steps.

A) run it locally in a container, B) push the container image to azure, using azure cli instructions. C) automate the full build and deployment using multistage CI/CD pipeline in GitHub Actions

Model Context Protocol (MCP)

MCP is an open protocol, developed by Anthrop\c, that standardizes how applications provide context to LLMs.

MCP Client
- Invokes **Tools**
- Queries for **Resources**
- Interpolates **Prompts**

MCP Server
- Exposes **Tools**
- Exposes **Resources**
- Exposes **Prompts**

Tools	Resources	Prompts
Model-controlled	*Application-controlled*	*User-controlled*
Functions invoked by the model	Data exposed to the application	Pre-defined templates for AI interactions
Retrieve / search	Files	Document Q&A
Send a message	Database records	Transcript Summary
Update DB records	API Responses	Output as JSON

Understanding MCP Tool Discovery

Dynamic Tool Discovery

- a mechanism that allows an AI agent **to discover available external tools** without needing hardcoded knowledge of each one.

MCP.Tool Decorator

- An MCP server hosts a set of functions that are exposed as tools using the @mcp.tool decorator.
- The **MCP server** hosts available tools.
- The **MCP client** dynamically discovers the tools.

MCP Server Scenarios

- GitHub
- Azure / Microsoft Learn Docs
- AWS
- Alibaba
- Atlassian
- Circle CI
- GitGuardian
- Playwright
- Twilio
- ...

Demo-MCP

14:57 Demonstration: Extend GitHub Copilot Agent Mode with MCP Servers

Demonstration

Extend GitHub Copilot Agent Mode with MCP Servers

In this demonstration, you'll see how to:

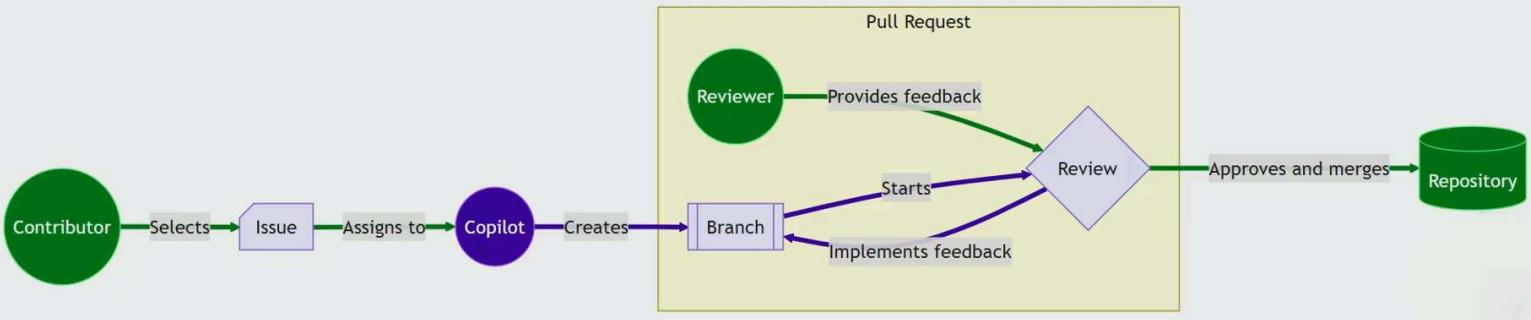
- **Add Azure / Microsoft Learn Docs MCP Servers**
- **Use MCP Server integration in GitHub Copilot Agent Mode**

GitHub Copilot Coding Agent

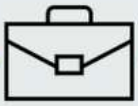
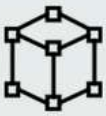
Feature	Copilot in the editor	Copilot coding agent
Interface	Your code editor	Issues and Pull Requests
Work Scope	Local files	Repository
Activation	Inline code suggestions, chat	Issue assignment
Customization	Custom instructions	Custom instructions
MCP Support	Yes	Yes
Vibe Coding	😎	😎

Ask Mode	Agent Mode
Line-by-line suggestions	**Complete project creation**
Manual file management	**Automatic project structure**
User drives each step	**Autonomous task execution**
Code completion	**End-to-end solutions**
Reactive assistance	**Proactive problem solving**
Single-file focus	**Multi-file coordination**

GitHub Copilot Coding Agent: Example



GitHub Copilot Coding Agent


Agent Mode: Purpose

- Enables Copilot to act as an **autonomous agent** that plans, executes, and iterates across multi-step development and DevOps tasks.
- Moves beyond single prompt–response interactions to **goal-oriented outcomes** across repositories, pipelines, and workflows.

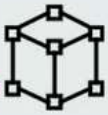
Agent Mode: Key Benefits

- **Reduced manual effort** by automating complex, multi-step tasks such as debugging, refactoring, and CI/CD troubleshooting.
- **Improved velocity** Keeping teams in flow while Copilot handles orchestration and analysis.
- **More consistent outcomes** policy-aware, context-driven execution aligned with repository and organization standards.

Agent Mode: Core Characteristics

- **Goal-driven execution**: Translates high-level intents into structured plans and actionable steps.
- **Context-aware reasoning**: Uses repository state, issues, pull requests, and workflows as live context.
- **Iterative feedback loop**: Observes results, adapts actions, and refines solutions automatically.

Use GitHub Copilot with AGENT.MD


Conceptual Overview

- Provides **persistent instructions** that guide the agent's decision-making, priorities, and scope, allowing teams to encode expectations directly alongside their codebase rather than relying on one-off prompts.

Use Case

- teams can describe **architectural conventions, coding standards, preferred tooling, and workflow rules** that the agent should consistently follow.
- This ensures that when Copilot plans and executes multi-step tasks, its actions **align with existing repository practices and organizational guidelines** rather than applying generic defaults.

Benefits

- **more predictable and repeatable automation** by shaping how the agent interprets goals.
- turns Copilot Agent Mode into a **customizable, policy-aware collaborator**. It allows teams to scale autonomous assistance safely, maintain consistency across contributors, and evolve agent behavior over time as standards, architectures, and workflows change

Demo-Coding Agent

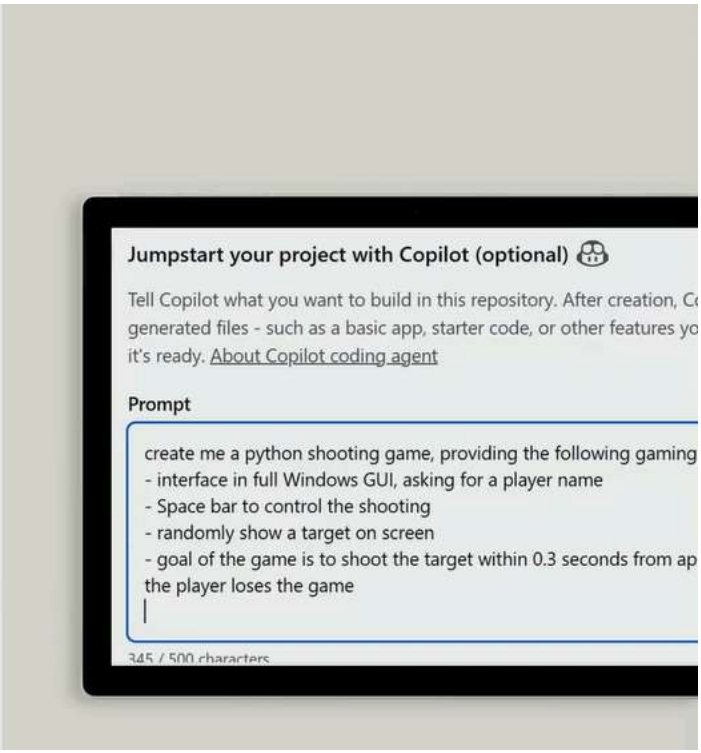
27:34 Demonstration: GitHub Copilot Coding Agent

Demonstration

GitHub Copilot Coding Agent

In this demonstration, you'll see how to:

- **Leverage GitHub Copilot Coding Agent for Issue Handling**
- **Use GitHub Copilot Coding Agent for App Development**
- **AGENT.md concepts**



Summary

Summary

In this module, you learned about using more advanced GitHub Copilot features, especially focused on Agent Mode. You learned about the integration of additional knowledge into the Agent, using Model Context Protocol (MCP) and the concept of Agent.md. Next, we shared details on several GitHub Copilot DevOps use cases, primarily PR Reviewing and Actions pipeline troubleshooting.

In this module, we covered:

- Understanding the core capabilities of GitHub Copilot Agent Mode
- Integration of additional knowledge by adding MCP Server connections to your Agents
- Configuration and usage of Agent.md
- PR Reviews using GitHub Copilot
- GitHub Actions Pipeline Troubleshooter

THE END